

LAFC-Evict: A Large-Scale Counterfactual Benchmark for Learned Cache Eviction: [Experiments & Analysis]

Anonymous Author(s)

ABSTRACT

Learned cache-eviction research has outgrown small ad hoc evaluations, yet empirical comparison remains difficult because prior studies often rely on private traces, release-incomplete preprocessing pipelines, or supervision targets that are not comparable across papers. This paper presents LAFC-Evict, a benchmark package for learned cache eviction in caching systems built around decision-aligned counterfactual supervision. Each canonical candidate row represents one candidate victim considered at one full-cache miss decision and includes release-facing provenance fields, cache-state features, and a finite-horizon counterfactual label. The canonical loss label, y_{loss} , is the continuation-policy miss cost over a finite horizon after forcing that candidate eviction. The corresponding value label, y_{value} , is defined as $-y_{\text{loss}}$. The benchmark also exposes decision-level and pairwise benchmark views to support value prediction, best-candidate selection, pairwise preference learning, and label-surface characterization.

The current open-release candidate covers five selected trace families—cloudphysics, metacdn, metakv, twemcache, and wiki2018—and the preserved release metadata currently reports 277,995,072 candidate rows, 2,363,286 decision-view rows, and a capped pairwise sample of 1,000,000 rows. Two additional families, brightkite and citibike, remain excluded from the current open subset. Beyond introducing the benchmark design, the paper describes the release schema, validation checks, and reproducibility surface needed for benchmark reuse, and it outlines benchmark tasks for value prediction, best-candidate selection, pairwise preference modeling, and descriptive label characterization. Full release validation for the preserved open candidate passed on June 29, 2026, so the present draft focuses on benchmark definition, release validation, metadata-backed characterization, and light pairwise sanity checks. The labels should nevertheless be interpreted as decision-aligned finite-horizon counterfactual targets rather than offline-optimal supervision.

CCS CONCEPTS

• **Information systems** → **Data management systems**; • **Computing methodologies** → *Machine learning*.

KEYWORDS

learned cache eviction, benchmark datasets, counterfactual supervision, cache replacement

1 INTRODUCTION

This paper presents LAFC-Evict, a benchmark for learned cache eviction built from real-trace-derived cache decisions and computed finite-horizon counterfactual labels. The contribution is not a new

eviction policy. It is a reusable benchmark surface for caching research that exposes candidate-level supervision, decision-level summaries, a shipped pairwise sample, and release-oriented metadata and validation. The motivating gap is now familiar in learned systems work: policy papers often compare methods under different traces, different preprocessing conventions, different supervision targets, and different release practices, which makes cross-paper empirical claims harder to interpret than the headline numbers suggest.

LAFC-Evict is designed to address that gap by making the supervised unit of comparison explicit. Each canonical candidate row corresponds to one candidate victim at one full-cache eviction decision, with decision metadata, candidate features, and a continuation-policy-specific finite-horizon counterfactual label. The release names that label y_{loss} , defines y_{value} as $-y_{\text{loss}}$, and treats both as properties of a candidate within a decision rather than as properties of a fully optimized future policy. From the same underlying release, the benchmark also exposes a decision view and a pairwise sample so that value prediction, within-decision best-candidate selection, pairwise preference prediction, and label-distribution characterization can be studied under a shared artifact. In that sense, LAFC-Evict is not merely a raw trace repository either: it sits above the trace layer and packages a decision-aligned supervision layer for benchmark reuse.

The current preserved open release is already large enough to make that benchmark framing substantive. Table 1 reports manifest-backed release scale of 277,995,072 candidate rows, 2,363,286 decision-view rows, and a shipped pairwise sample of 1,000,000 rows across five selected trace families. Table 8 summarizes the decision view by family, capacity, horizon, and split. Table 9 shows that decision complexity is nontrivial even before any model training: in the decision view, candidate counts have minimum 32, median 64, mean 117.63, and maximum 256. Table 5 reports coverage of 118,635 unique decisions in the pairwise sample. These numbers are useful for scoping the benchmark, and the preserved release passed full real-release validation on June 29, 2026. They should still be read conservatively, however, as release descriptors and decision-level summaries rather than as full row-level characterization of labels.

The release also makes a deliberate boundary between raw data provenance and benchmark packaging. LAFC-Evict does not claim authorship of the underlying traces, and the open subset is intentionally narrower than the broader internally generated material. Instead, the benchmark contribution is the releaseable supervision artifact derived from those traces: the schema, the task views, the manifest, the checksums, and the validation machinery needed for reuse. That separation matters both scientifically, because the labels are finite-horizon and continuation-policy-specific decision

targets rather than offline-optimal targets, and operationally, because redistribution constraints affect what can responsibly appear in the open subset.

To our knowledge, existing public cache-replacement artifacts do not provide a reusable candidate-level counterfactual supervision table at this scale. The novelty claim is therefore about the released supervision artifact, schema, validation surface, and benchmark tasks rather than about a new eviction policy.

The paper therefore makes a benchmark claim rather than a deployment claim. First, learned cache eviction can be studied through decision-aligned candidate supervision instead of only through end-policy comparisons. Second, one release can support multiple related tasks through candidate, decision, and pairwise views. Third, release engineering and validation are part of the scientific contribution because they determine whether later work can reproduce or extend the benchmark. Fourth, a credible SIGMOD Experiments & Analysis paper can focus on benchmark construction, release structure, characterization, and simple baselines without overstating downstream systems impact or claiming model coverage beyond the evidence reported here.

The rest of the paper proceeds as follows. Section 2 reviews the learned cache-eviction setting and motivates decision-aligned supervision. Section 3 defines the benchmark scope and release-governance boundary. Sections 4 and 5 describe the label definition, schema, and derived views. Section 6 explains release construction and validation. Sections 7 and 8 define the benchmark tasks and the current characterization surface. Section 9 covers limitations, ethics, and reuse constraints, followed by related work and conclusion.

2 BACKGROUND AND MOTIVATION: LEARNED CACHE EVICTION

At each full-cache miss, a caching system must evict one resident object before admitting the requested one. Classical policies such as LRU operationalize this choice through simple recency structure, while learned or learning-augmented approaches attempt to predict which eviction will lead to lower future miss cost. In practice, these approaches differ in feature sets, prediction targets, and evaluation methodology.

The main difficulty for a benchmark is that victim selection is structured, not i.i.d. A model does not act on one object in isolation; it acts within a decision set whose size, feature spread, and ambiguity vary across traces and capacities. A useful benchmark should therefore preserve decision boundaries instead of flattening them away.

This observation motivates the core LAFC-Evict design choice: represent supervision at the candidate-within-decision level. Each row corresponds to one candidate victim at one full-cache eviction decision, and the label records the finite-horizon counterfactual miss cost of forcing that eviction under a fixed continuation policy. This representation allows both pointwise learning and within-decision evaluation, while keeping the supervised object aligned with the actual cache-eviction choice.

From that candidate-level foundation, the benchmark derives two secondary evaluation surfaces. Grouping all rows with the

same decision_id produces a decision view that records candidate count, optimal-set membership, tie structure, and regret summaries. Sampling candidate pairs from the same decision produces the shipped pairwise view, which turns within-decision comparisons into a simpler classification problem without discarding the original decision boundary.

A single full-cache miss can therefore appear in three complementary forms. If one decision exposes k admissible victims, the release records k candidate rows, one decision-group summary, and up to $k(k-1)/2$ within-decision pairwise comparisons before any sampling cap is applied. This draft keeps that task-view progression textual rather than introducing a second overview figure beside the main pipeline diagram.

3 BENCHMARK DESIGN

LAFC-Evict is organized around five layers: external raw traces, processed traces, generated features, generated labels, and benchmark tasks. The release in this repository begins after candidate-row generation and focuses on converting existing generated data into a reusable benchmark package for cache-eviction benchmarking, with schema, validation, and release metadata.

The current preserved open subset is `lafe-evict-v0.1-open`. The preserved current-contract release metadata identifies five selected families: `cloudphysics`, `metacdn`, `metakv`, `twemcache`, and `wiki2018`. The family registry also lists `brightkite` and `citibike` as excluded from the current open subset for redistribution reasons. This scope boundary is a release-governance decision, not a statement that the broader internally generated dataset is automatically redistributable. Not every selected family appears in every split in the current preserved release; for example, `metacdn` contributes train and validation decision rows but no test decision rows.

The canonical benchmark unit is the candidate row. A single row corresponds to one candidate victim at one full-cache miss decision, so one eviction decision with $|C_t|$ admissible victim choices contributes $|C_t|$ candidate rows. Decision-level and pairwise benchmark objects are derived from that candidate-level base rather than collected independently. This design keeps the benchmark centered on cache-eviction decisions instead of conflating the benchmark contribution with end-to-end deployment claims.

The benchmark is designed to satisfy four requirements:

- (1) **Decision alignment.** Preserve within-decision structure and candidate competition.
- (2) **Task multiplicity.** Support pointwise, decision-level, and pairwise learning tasks from one release.
- (3) **Releaseability.** Separate public benchmark artifacts from upstream raw-trace ownership and hosting constraints.
- (4) **Validation.** Expose checks for schema, consistency, family scope, and checksums.

4 COUNTERFACTUAL LABEL GENERATION

The benchmark label is defined on candidate victims rather than on completed policies. Let $r_1, r_2, \dots$ denote a request sequence, and let t index a request r_t that misses when the cache is full. Write S_t^- for the cache state immediately before the eviction choice and $C_t \subseteq S_t^-$ for the candidate set of resident objects that may be evicted at that decision. For any candidate $c \in C_t$, consider the intervention

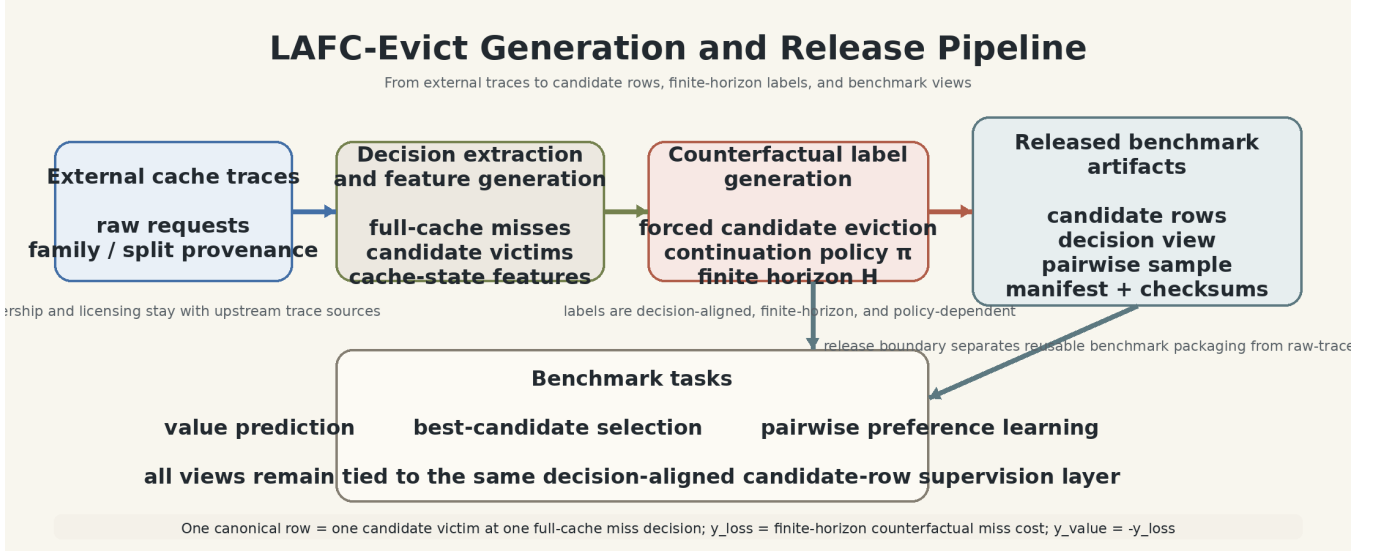


Figure 1: LAFC-Evict generation and release pipeline. The benchmark converts external cache traces into decision-level candidate rows, finite-horizon counterfactual labels, and released benchmark views for value prediction, best-candidate selection, and pairwise preference learning.

Table 1: Dataset scale from preserved-release metadata. Counts are manifest-backed or view-backed as noted.

metric	value
candidate rows	277,995,072
decision rows	2,363,286
pairwise-sample rows	1,000,000
preserved release size	2.7G
candidate parquet files	168
total release files	176
manifest file_inventory count	176
checksum entry count	175
selected trace families	cloudphysics, metacdn, metakv, twemcache, wiki2018
excluded trace families	brightkite, citibike
full validation status	passed

Table 2: Split counts from manifest-backed candidate-row metadata. These are candidate-row counts, not decision-view counts.

split	row_count
test	28,205,952
train	205,726,368
val	44,062,752

that evicts c , admits the missed request, and then follows a fixed continuation policy π for the next H requests. The resulting finite-horizon counterfactual loss is

$$L_{t,H}^{\pi}(c) = \#\{\tau \in \{t+1, \dots, t+H\} : r_{\tau} \text{ misses in the induced trajectory}\}.$$

The release stores this quantity as y_{loss} , and it stores the corresponding value label as $y_{\text{value}} = -y_{\text{loss}}$. The triggering miss at

time t is common to all candidates and is therefore not the decision-discriminating part of the label.

This label has two consequences for the paper. First, it creates a common numerical target for regression-style benchmark tasks. Second, it induces structured decision-level objects. For a fixed decision (t, S_t^-, C_t) , the benchmark-optimal candidate set is

$$C_t^* = \arg \min_{c \in C_t} L_{t,H}^{\pi}(c),$$

and the per-candidate regret is

$$\Delta_{t,H}^{\pi}(c) = L_{t,H}^{\pi}(c) - \min_{c' \in C_t} L_{t,H}^{\pi}(c').$$

These quantities define best-candidate selection targets, pairwise preferences, and tie structure within a decision.

The benchmark should describe this supervision carefully. It is a generated counterfactual label family, not an upstream annotation. It is finite-horizon, not asymptotic. It is tied to a continuation policy, not a universally optimal control objective. The label is therefore decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label in the sense of solving the full future control problem. These boundaries are central to the scientific positioning of the paper.

5 DATASET SCHEMA AND VIEWS

The canonical candidate schema currently consists of decision metadata, candidate identity, split information, labels, and feature columns. Each row corresponds to exactly one candidate victim at exactly one full-cache miss decision. Trace identity and split metadata are recorded through `trace_name`, `trace_family`, `dataset_source`, and `split`. Cache configuration is recorded through `capacity` and `horizon`. Decision identifiers include `decision_id` and `decision_t`. The schema also records a per-decision chunk ID. That field is named `decision_chunk_id`. Candidate identity is represented through

Table 3: Compact summary of candidate-row schema groups and derived-view fields.

Column group	Notes
Decision metadata	family, capacity, horizon, split, IDs
Candidate identity	candidate page ID
Labels	y_loss, y_value
Feature groups	request, candidate, cache, disagreement, rate
Derived decision fields	tie and regret summaries
Derived pairwise fields	A/B labels and regret differences

the field `candidate_page_id`. The labels are `y_loss`, the finite-horizon counterfactual miss cost after forcing that candidate eviction under the fixed continuation policy, and `y_value` = $-y_{\text{loss}}$. Feature columns cover request context, candidate context, predictor/LRU signals, cache summary statistics, and recent request or hit rates.

Beyond candidate rows, the release defines two derived views. The *decision view* aggregates all rows that share a decision identifier and summarizes each decision with candidate count, minimum loss, maximum value, the set of optimal candidate IDs, tie count, and regret summary statistics. The *pairwise view* compares candidate pairs from the same decision and derives pairwise preference labels from the ordering of their `y_loss` values, with ties when the losses are equal. The current preserved open release does not ship the full pairwise view; it ships a capped pairwise sample of 1,000,000 rows instead.

Operationally, the schema can be read as a layered projection over the same decision events: candidate rows are the atomic supervised objects, the decision view groups those rows by `decision_id`, and the shipped pairwise sample keeps only selected within-decision comparisons. Table 3 summarizes those column groups and derived fields directly in manuscript form.

6 RELEASE CONSTRUCTION AND VALIDATION

The release pipeline is designed to package existing generated candidate rows into a benchmark artifact without requiring raw traces in the repository. The builder uses an out-of-core DuckDB path to ingest candidate shards, normalize split values, emit partitioned Parquet candidate rows, derive the decision view, optionally create a capped pairwise sample, and write release metadata and checksums.

The validation surface is part of the benchmark contribution. Current checks include:

- schema completeness for candidate partitions,
- split normalization and split-set consistency,
- presence of selected families and absence of blocked families,
- duplicate candidate detection within decision groups,
- consistency of decision metadata within a decision,
- decision-view schema and count consistency,
- checksum verification.

The preserved current-contract open release has passed lightweight metadata repair, repository tests, publication-bundle validation, and full real-release validation. The June 29, 2026 preserved-release validation run completed successfully with `exit_status=0`.

Table 4: Release artifact layout using release-relative paths for the preserved benchmark tree.

artifact	relative_path
candidate rows	data/candidate_rows/
decision view	data/decision_view/decision_view.parquet
pairwise sample	data/pairwise_sample/pairwise_sample.parquet
release manifest	metadata/release_manifest.json
validation report	metadata/validation_report.md
checksums	metadata/checksums.sha256

The validator CLI does not rewrite the release-internal validation-report markdown file, so that file still reflects earlier build-time paths and should not be treated as the authoritative June 29 validation record. For the anonymous SIGMOD submission, this means the manuscript can state that the preserved release passed end-to-end validation without relying on unpublished hosting or packaging details.

Table 4 summarizes the preserved release tree that the manuscript relies on: candidate rows, derived benchmark views, manifest, validation report, and checksums. Anonymous submission packaging is intentionally treated as external to that preserved release tree, so the paper’s empirical and validation claims rest only on the preserved artifact itself.

7 BENCHMARK TASKS AND BASELINES

LAFC-Evict supports four benchmark tasks from one release.

Candidate-level value prediction. Predict `y_loss` or `y_value` from a single candidate row. This is a pointwise task over the canonical benchmark unit and measures how well a model predicts finite-horizon counterfactual eviction quality for one candidate at one decision.

Best-candidate selection. Given all candidate rows for one decision, identify a member of the minimum-loss set. This task can be framed as ranking or decision-level classification and connects most directly to the actual cache-eviction choice because it evaluates whether a method selects a benchmark-optimal victim within that decision.

Pairwise preference prediction. Predict whether candidate A is better than candidate B within the same decision, where “better” means lower `y_loss` or equivalently higher `y_value`. This task uses the pairwise view or the shipped pairwise sample and provides a simpler learning interface for within-decision preference modeling. Because equal losses induce ties, any paper using this task should state explicitly whether evaluation includes tie rows, excludes them, or maps them to a separate class.

Candidate-label distribution characterization. Summarize the empirical label surface without training a predictive model. This characterization task studies distributions of `y_loss`, `y_value`, regret, tie frequency, and decision ambiguity across families, capacities, horizons, and splits. It is descriptive rather than competitive, but it is important for understanding benchmark difficulty and for interpreting downstream task results.

The empirical scope of this draft is intentionally narrow. Rather than presenting a large model bake-off, we use the released task

Table 5: Pairwise-sample-backed summary statistics for the shipped sample view.

statistic	value
pairwise rows	1,000,000
split breakdown	test=102,024; train=737,896; val=160,080
family breakdown	wiki2018=250,872; cloudphysics=245,336; metakv=190,584; twemcache=169,064; metacdn=144,144
unique decisions represented	118,635
label distribution	a_better=6,134; b_better=113,898; tie=879,968

Table 6: Pairwise-sample-backed light sanity baselines on the non-tie subset of the shipped pairwise sample. These are non-feature checks only because the shipped sample omits candidate-side predictor and LRU features.

baseline	rows	accuracy	log loss
majority_non_tie	120,032	0.9489	0.2023
random_non_tie	120,032	0.4980	0.6931

Table 7: Benchmark task definitions and representative evaluation outputs.

Task	Inputs	Representative outputs
Value regression	candidate rows	MAE, RMSE, and optional rank correlation between predicted and observed values
Best-candidate selection	decision groups	Top-1 accuracy against the benchmark-optimal set and mean realized regret relative to the best available candidate
Pairwise preference	pairwise rows from one decision	Accuracy or ROC-AUC, with tie handling stated explicitly and non-tie-only reporting marked when used
Label characterization	candidate or decision aggregates	Histograms, quantiles, tie rates, regret summaries, and per-family/capacity/horizon breakdowns

definitions to separate what the artifact already supports reproducibly from what would require additional candidate-row execution or feature joins. The manuscript therefore treats the current release primarily as a benchmark-definition and characterization contribution.

The only executed baseline results reported here are two non-feature sanity checks on the shipped pairwise sample: a majority baseline and a random baseline, both evaluated on the non-tie subset only. Because the shipped pairwise sample omits candidate-side predictor and LRU feature columns, these numbers test label balance and split behavior in the sample rather than the predictive power of cache-state features, and they should not be overread as end-to-end cache-policy evidence.

8 EMPIRICAL CHARACTERIZATION

The empirical character of gravity for this paper should be benchmark characterization, not an oversized model bake-off. The release metadata already supports several useful facts: total candidate rows, total decision rows, pairwise-sample size, split counts,

Table 8: Decision-view-backed benchmark composition by family, capacity, horizon, and split.

dimension	counts
trace_family	wiki2018=598,560; cloudphysics=576,219; metakv=454,053; twemcache=397,494; metacdn=336,960
capacity	32=613,314; 64=599,172; 128=582,678; 256=568,122
horizon	4=787,762; 8=787,762; 16=787,762
split	test=242,418; train=1,746,882; val=373,986

Table 9: Decision-view-backed candidate-count statistics. These values summarize the number of candidates per decision in the preserved release.

group	min	med	mean	max
overall	32	64	117.63	256
cloudphysics	32	64	118.35	256
metacdn	32	64	117.12	256
metakv	32	64	119.52	256
twemcache	32	64	111.52	256
wiki2018	32	64	119.85	256

Table 10: Decision-view-backed tie and regret summaries.

statistic	value
total decisions	2,363,286
decisions with tie_count > 1	2,363,286
fraction with tie_count > 1	1
max tie_count	256
regret_mean mean/std/max	0.01 / 0.02 / 0.88
regret_std mean/std/max	0.05 / 0.08 / 1.24
regret_max mean/std/max	0.32 / 0.47 / 4

family counts, capacity counts, and horizon counts. These are enough to establish the scale and heterogeneity of the benchmark in the current submission draft.

The present manuscript therefore limits characterization claims to metadata-backed, decision-view-backed, and pairwise-sample-backed summaries. Detailed row-level distributions of `y_loss` and `y_value` require a candidate-row scan and are intentionally omitted here rather than approximated from incomplete metadata.

9 LIMITATIONS, ETHICS, AND REUSE

This benchmark should be interpreted with several constraints in mind. First, the main label is a finite-horizon continuation-policy-specific counterfactual target. It is decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label for the full future control problem. Good performance on this label therefore does not automatically imply online deployment gains.

Second, the labels depend on the continuation policy used after the forced eviction. A different continuation policy or a different horizon can change the numerical target, the optimal set within a decision, and the induced pairwise preferences. Comparisons should therefore be made only across artifacts that keep these label-generation choices fixed.

Third, the open subset is intentionally narrower than the larger internally generated dataset mix. Two trace families remain excluded from the current open subset for redistribution reasons, and not every selected family appears in every split in the preserved release. This release-governance boundary is part of the benchmark story and should be made explicit rather than hidden in artifact packaging details.

Fourth, benchmark reuse should preserve provenance distinctions. The release exposes generated supervision and derived tasks, but it does not erase the attribution, citation, or licensing requirements of the upstream raw traces from which candidate rows were derived. Those raw traces remain external artifacts with their own governance constraints.

Fifth, the preserved open release should be distinguished from any eventual public distribution channel. Lightweight metadata repair, bundle validation, repository tests, and full real-release validation have passed, but this anonymous manuscript centers the preserved release tree and validation record rather than final hosting details. The paper should describe that boundary candidly rather than overstating release availability.

Finally, the generated labels are intended for offline benchmarking claims. They can support comparative modeling studies, but they should not be presented as automatic evidence that a learned policy is ready for operational deployment without separate online evaluation, systems integration, and safety review. The current manuscript also does not report full-policy evaluations against external eviction algorithms such as SIEVE, S3-FIFO, MAT, or Parrot; those methods are relevant comparison points and future benchmark consumers, but they are not claimed as evaluated baselines here.

10 RELATED WORK

10.1 Trace Datasets, Workload Studies, and Benchmark Surfaces

Public traces and benchmark infrastructure provide the substrate for cache evaluation. The UMass Trace Repository and large-scale workload studies of key-value stores and Twemcache clusters demonstrate the value of realistic, heterogeneous request streams for systems research [1, 25, 28]. More broadly, benchmark artifacts such as YCSB, OLTP-Bench, and the Join Order Benchmark made comparison easier by publishing reusable workloads, interfaces, and evaluation surfaces rather than only one-off experiments [5, 6, 14]. Existing trace libraries and simulators therefore supply essential raw material for cache studies, and some public artifacts even expose oracle-style features or procedural oracle policies. LAFC-Evict packages a different layer of supervision: for each full-cache miss decision and candidate victim, it releases finite-horizon counterfactual loss labels under a fixed continuation policy, together with decision-level and pairwise views.

Those distinctions matter because raw traces, workload studies, and simulator-oriented artifacts do not by themselves provide a reusable table of candidate-level finite-horizon counterfactual outcomes. LAFC-Evict is intended to complement that ecosystem rather than replace it. The artifact is useful precisely when

a downstream user wants to study offline value regression, best-candidate selection, preference learning, or label-distribution analysis without first regenerating the same counterfactual rollouts from scratch.

10.2 Learned Eviction and Imitation Learning

Recent learned-caching papers likewise contribute learned policies or learning frameworks rather than benchmark releases. RL-Cache studies learned cache admission for content delivery [13]. Parrot uses imitation learning to approximate Belady-style replacement decisions from past accesses [16], which makes it especially relevant as a Belady-imitation reference point. MAT focuses on reducing the overhead of ML-guided eviction by using a heuristic filter around a learned module [27]. ALPS emphasizes lightweight learned replacement and the practical cost of learning overheads [22], while Feedforward Neural Networks for Caching is useful precisely because it questions whether heavier learned models are always justified [9]. HR-Cache extends the learned-caching line into edge delivery settings [24]. Across these works, the public contribution is still mainly a policy or policy-learning framework. To our knowledge, we are not aware of a public artifact in this line that releases a static dataset of per-candidate finite-horizon counterfactual eviction labels as a reusable benchmark surface.

10.3 Modern Eviction Policies

Classical cache-replacement work defines the policy landscape that any learned benchmark must eventually interface with. Belady's optimal look-ahead policy remains the canonical offline reference point [2]. Practical online policies then emphasize different signals: LIRS focuses on inter-reference recency [11], ARC adaptively balances recency and frequency [19], CLOCK-Pro revisits CLOCK-style replacement with stronger reuse sensitivity [10], and GreedyDual-Size accounts for cost and size [3]. TinyLFU and W-TinyLFU further show that admission and segmented cache management matter in modern software caches [7, 8]. More recent object-cache work shows that policy progress does not always come from heavier machinery: S3-FIFO argues that simple FIFO-based queueing can beat more elaborate baselines at scale [29], while SIEVE emphasizes turn-key deployability for web caches [30]. These works are important baselines and deployment reference points, but they remain policy artifacts rather than supervised-label releases.

10.4 Learning-Augmented Caching and Predictive Advice

Another relevant line of work treats caching as an online problem augmented by predictions rather than as a supervised dataset. Competitive Caching with Machine Learned Advice and later learning-augmented caching results formalize how predictive advice can improve classical online algorithms while preserving robustness guarantees [18, 26]. This perspective is close in spirit because it also separates policy decisions from predictive signals, but the objects of study differ. Learning-augmented caching papers typically reason about next-arrival-style advice and competitive guarantees for online eviction algorithms. By contrast, LAFC-Evict

releases finite-horizon, continuation-policy-dependent counterfactual labels for every candidate within a decision. The artifact supports value regression, best-candidate selection, pairwise preference learning, and label characterization from one release, rather than prescribing one online policy with one advice interface.

10.5 Benchmark and Dataset Positioning

Public learning-to-rank collections such as LETOR and the Yahoo! Learning to Rank Challenge show how reusable labeled artifacts can organize a research community [4, 17, 20]. The label design in LAFc-Evict also connects to counterfactual learning and ranking because each cache decision induces a candidate set, an optimal subset, regret summaries, and a natural pairwise structure [12, 15, 21, 23]. Even so, LAFc-Evict is not simply a ranking dataset transplanted into systems. Its labels are tied to finite-horizon eviction outcomes under a specified continuation policy, and they are not globally optimal online decisions. The central differentiation to maintain throughout is therefore cautious but concrete: to our knowledge, we are not aware of a public cache-eviction artifact that releases reusable candidate-level finite-horizon counterfactual supervision together with decision-level and pairwise task views at this scale. In that sense, LAFc-Evict is not a new policy and not merely a raw trace repository, but a reusable benchmark artifact for learned cache-eviction research that complements existing traces, simulators, and policy papers.

11 CONCLUSION

This draft positions LAFc-Evict as a benchmark paper for learned cache eviction rather than as a new policy paper. The core contribution is a release-oriented benchmark package built around candidate-level counterfactual supervision, derived decision and pairwise tasks, and validation-aware artifact construction. The current preserved open release metadata already indicates benchmark scale large enough to justify a benchmark paper, while the remaining work is concentrated in final validation, anonymous artifact packaging, and the minimum baseline and characterization jobs needed for a credible SIGMOD submission.

The immediate path forward is narrow and practical: finish the statistics extraction needed for the main tables, implement the small baseline suite, anonymize the artifact surface, and run final release validation only when explicitly approved. If that path is executed cleanly, LAFc-Evict can support a compact but defensible Experiment & Analysis paper in the Benchmarks and Datasets category.

REFERENCES

- [1] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload Analysis of a Large-Scale Key-Value Store. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 53–64.
- [2] L. A. Belady. 1966. A Study of Replacement Algorithms for a Virtual-Storage Computer. *IBM Systems Journal* 5, 2 (1966), 78–101. <https://doi.org/10.1147/sj.52.0078>
- [3] Pei Cao and Sandy Irani. 1997. Cost-Aware WWW Proxy Caching Algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems*. 193–206. <https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms>
- [4] Olivier Chapelle, Yi Chang, and Tie-Yan Liu (Eds.). 2011. *Proceedings of the Yahoo! Learning to Rank Challenge*. JMLR Workshop and Conference Proceedings, Vol. 14. JMLR.org. <https://proceedings.mlr.press/v14/>
- [5] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the ACM Symposium on Cloud Computing*. 143–154.
- [6] Djellal Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. 2013. OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proceedings of the VLDB Endowment* 7, 4 (2013), 277–288.
- [7] Gil Einziger, Ohad Eytan, Roy Friedman, and Ben Manes. 2018. Adaptive Software Cache Management. In *Proceedings of the 19th International Middleware Conference*. 94–106. <https://doi.org/10.1145/3274808.3274816>
- [8] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage* 13, 4 (2017), 35:1–35:31. <https://doi.org/10.1145/3149371>
- [9] Vladyslav Fedchenko, Giovanni Neglia, and Bruno Ribeiro. 2018. Feedforward Neural Networks for Caching: Enough or Too Much? *SIGMETRICS Performance Evaluation Review* 46, 3 (2018), 139–142. <https://doi.org/10.1145/3308897.3308958>
- [10] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. CLOCK-Pro: An Effective Improvement of the CLOCK Replacement. In *Proceedings of the Annual USENIX Technical Conference*. 323–336. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>
- [11] Song Jiang and Xiaodong Zhang. 2002. LIRS: An Efficient Low Inter-Reference Recency Set Replacement Policy to Improve Buffer Cache Performance. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 31–42. <https://doi.org/10.1145/511334.511340>
- [12] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased Learning-to-Rank with Biased Feedback. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining*. 781–789.
- [13] Vadim Kirilin, Aditya Sundararajan, Sergey Gorinsky, and Ramesh K. Sitaraman. 2020. RL-Cache: Learning-Based Cache Admission for Content Delivery. *IEEE Journal on Selected Areas in Communications* 38, 10 (2020), 2372–2385. <https://doi.org/10.1109/JSAC.2020.3000415>
- [14] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *The VLDB Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>
- [15] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. 2011. Unbiased Off-line Evaluation of Contextual-Bandit-Based News Article Recommendation Algorithms. In *Proceedings of the 4th ACM International Conference on Web Search and Data Mining*. 297–306. <https://doi.org/10.1145/1935826.1935878>
- [16] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An Imitation Learning Approach for Cache Replacement. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 6237–6247. <https://proceedings.mlr.press/v119/liu20f.html>
- [17] Tie-Yan Liu. 2009. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval* 3, 3 (2009), 225–331. <https://doi.org/10.1561/1500000016>
- [18] Thodoris Lykouris and Sergei Vassilvitskii. 2021. Competitive Caching with Machine Learned Advice. *J. ACM* 68, 4 (2021), 24:1–24:25. <https://doi.org/10.1145/3447579>
- [19] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. 115–130. <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [20] Tao Qin, Tie-Yan Liu, Jun Xu, and Hang Li. 2010. LETOR: A Benchmark Collection for Research on Learning to Rank for Information Retrieval. *Information Retrieval* 13, 4 (2010), 346–374. <https://doi.org/10.1007/s10791-009-9123-y>
- [21] Yuta Saito. 2020. Unbiased Pairwise Learning from Biased Implicit Feedback. In *Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval*. 5–12. <https://doi.org/10.1145/3409256.3409812>
- [22] Rana Shahout and Roy Friedman. 2024. Sketching the Path to Efficiency: Lightweight Learned Cache Replacement. In *27th International Conference on Principles of Distributed Systems (OPODIS 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 286)*. 34:1–34:21. <https://doi.org/10.4230/LIPIcs.OPODIS.2023.34>
- [23] Adith Swaminathan and Thorsten Joachims. 2015. Counterfactual Risk Minimization: Learning from Logged Bandit Feedback. In *Proceedings of the 32nd International Conference on Machine Learning*. 814–823. <https://proceedings.mlr.press/v37/swaminathan15.html>
- [24] Hoda Torabi, Hamzeh Khazaei, and Marin Litoiu. 2024. A Learning-Based Caching Mechanism for Edge Content Delivery. In *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. 236–246. <https://doi.org/10.1145/3629526.3645037>
- [25] University of Massachusetts Amherst. 2025. UMass Trace Repository. <https://traces.cs.umass.edu/> Official trace repository website.
- [26] Alexander Wei. 2020. Better and Simpler Learning-Augmented Online Caching. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms*

- and Techniques (APPROX/RANDOM 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 176). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 60:1–60:17. <https://doi.org/10.4230/LIPIcs.APPROX/RANDOM.2020.60>
- [27] Dongsheng Yang, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2023. A Learned Cache Eviction Framework with Minimal Overhead. <https://doi.org/10.48550/arXiv.2301.11886> arXiv:2301.11886 [cs.OS]
- [28] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A Large-Scale Analysis of Hundreds of In-Memory Cache Clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 191–208. <https://www.usenix.org/conference/osdi20/presentation/yang>
- [29] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO Queues Are All You Need for Cache Eviction. In *ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP '23)*. <https://doi.org/10.1145/3600006.3613147>
- [30] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 1229–1246. <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>